

National University of Singapore
School of Computing

Semester 1, AY2026-27

CS4246/CS5446

Reinforcement Learning and Sequential Decision Making

Issued: 26 Aug 2026

Tutorial 4 Week 6: Function Approximation and Policy Search

Guidelines

You may discuss the content of the questions with your classmates. But everyone should work on and be ready to present ALL the solutions.

Problem 1: Review

Briefly explain the following concepts.

- (a) **Linear regression and loss-function optimization.**
- (b) **Gradient descent.**
- (c) **Deep learning in neural networks.**

Problem 2: Approximating TD Learning

In a navigation task, the agent's state is its position $s = (x, y)$, and the goal is at (x_g, y_g) . When the set of possible positions is large or continuous, storing a separate utility for every state is impractical. We therefore approximate state utility using the coordinates and the Euclidean distance to the goal as features. After observing a transition (s, r, s') , the agent applies one-step TD learning with learning rate α and discount factor γ .

Write out the parameter update equations for temporal-difference (TD) learning with

$$\hat{U}_\theta(x, y) = \theta_0 + \theta_1 x + \theta_2 y + \theta_3 \sqrt{(x - x_g)^2 + (y - y_g)^2}.$$

Problem 3: Approximating Q-Learning

Q-learning is a model-free, off-policy TD control method. It does not need a model of the environment's transition dynamics. Instead, each observed transition (s, a, r, s') is used to update the sampled state-action pair toward a target containing the immediate reward and the best estimated action value at the next state. This problem first applies the tabular update, then replaces the table with a simple function approximator.

Consider a system with a single state variable $x \in \{0, 1\}$ and actions a_1 and a_2 . An agent observes the state and reward. Assume $\gamma = 0.9$.

(a) Perform two steps of tabular Q-learning. Use $\alpha = 0.5$ and initialize all entries to zero. Show the table after each step.

Use

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

(i) First transition: $x = 0$, reward $r = 10$, action a_1 , next state $x' = 1$.

(ii) Second transition: $x = 1$, reward $r = -5$, action a_2 , next state $x' = 0$.

(b) Now use $Q(x, a_1) = \beta_1 x$ and $Q(x, a_2) = \beta_2 x$. Let $\alpha = 0.5$ and initialize $\beta_1 = \beta_2 = 0$. Show the parameters after each step.

Use

$$\beta_i \leftarrow \beta_i + \alpha \delta \frac{\partial Q(x, a)}{\partial \beta_i}, \quad \delta = r + \gamma \max_{a'} Q(x', a') - Q(x, a).$$

(i) First transition: $x = 1$, reward $r = 10$, action a_1 , next state $x' = 1$.

(ii) Second transition: $x = 1$, reward $r = -5$, action a_2 , next state $x' = 0$.

(c) After enough data is observed, which method should perform better here? Why? How can the poorer representation be improved?

Problem 4: Persy the Mars Rover Learns by REINFORCE

Persy is a Mars rover that must learn to navigate from camera observations without being given the correct action for each image. At each time step, it observes a camera state s and samples a drive command (forward, left, right, or stop). The environment supplies rewards, and the return from an episode indicates how well the resulting trajectory performed. Persy's stochastic softmax policy $\pi_\theta(a \mid s)$, parameterised by network weights θ , gives the probability of choosing action a in state s . It is trained using REINFORCE, a Monte Carlo policy-gradient algorithm that learns from completed episodes.

Let r_t be the reward received after taking action a_t in state s_t . For an episode with T actions, define the discounted return by

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k,$$

where γ is the discount factor.

(a) Why does REINFORCE use $\nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t$? Explain each part.

(b) Baseline and state value. A baseline $b(s_t)$ is a state-dependent reference value that does not depend on the action sampled at that state. REINFORCE can use $G_t - b(s_t)$ in place of G_t to weight the policy gradient. The state-value function $V_\pi(s)$ is the expected discounted return starting from state s and following the current policy $\pi = \pi_\theta$. Thus G_t is the return from one sampled episode, whereas $V_\pi(s_t)$ is an average over possible future trajectories.

How does subtracting a baseline help, and why is $b(s_t) = V_\pi(s_t)$ a good choice?

(c) An episode terminates after three actions, with rewards $r_0 = 1$, $r_1 = 0$, $r_2 = 2$, and $\gamma = 0.9$. Compute G_0 and G_1 .

Problem 5: PPO and the Spacecraft Attitude Controller

A spacecraft controller is trained with PPO. Let

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

(a) **Policy ratio meaning.** What does the probability ratio $r_t(\theta)$ mean?

(b) **Advantage sign.** What does the sign of the estimated advantage $\hat{A}_t$ mean?

(c) **Clipping calculation.** Let $\epsilon = 0.2$ and $r_t = 1.3$. Compute $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$.

(d) **Objective calculation.** Now let $\hat{A}_t = 2.0$. Compute the unclipped and clipped terms

$$u = r_t \hat{A}_t, \quad c = \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t.$$

Then compute the per-sample PPO objective $L_t^{\text{clip}} = \min(u, c)$. The minimum is taken after multiplying by the advantage, whether $\hat{A}_t$ is positive or negative.

(e) **Conceptual benefit.** How does PPO clipping compare with the KL-divergence constraint in TRPO?
